R Niveau I — Exercices Jour 2

Module 5 : Import et exploration de données

Exercice 5.1 : Importer un fichier CSV [Difficulté : Facile]

Énoncé :

1. Créez un petit fichier CSV en R avec `write.csv()` contenant des données de ventes :
    - produit : c("Pomme", "Banane", "Orange", "Fraise", "Kiwi")
    - prix : c(2.50, 1.80, 3.20, 4.50, 2.90)
    - quantite : c(100, 150, 80, 60, 120)
2. Réimportez ce fichier avec `read.csv()`
3. Vérifiez la structure avec `str()`
4. Essayez `read.csv2()` et expliquez la différence

Indice : `write.csv()` écrit un CSV avec `,` comme séparateur. `read.csv2()` utilise `;` comme séparateur (format européen). Pensez à `row.names = FALSE` pour éviter les numéros de ligne.

▶ Solution

Exercice 5.2 : Importer avec des options avancées [Difficulté : Intermédiaire]

Énoncé :

1. Créez manuellement un fichier texte (avec `writeLines()`) contenant des données séparées par des tabulations, avec un en-tête et quelques commentaires (lignes commençant par #)
2. Importez-le avec `read.table()` en spécifiant les bons paramètres

3. Importez seulement les 3 premières lignes de données avec `nrows`
4. Spécifiez manuellement les types de colonnes avec `colClasses`

Indice : `read.table()` est la fonction la plus flexible. Paramètres utiles : `sep`, `header`, `comment.char`, `skip`, `nrows`, `colClasses`.

▶ Solution

Exercice 5.3 : Explorer un jeu de données intégré [Difficulté : Facile]

Énoncé : Chargez le jeu de données `mtcars` (intégré dans R) et explorez-le :

1. Affichez les 6 premières et 6 dernières lignes
2. Affichez les dimensions (lignes x colonnes)
3. Affichez la structure complète
4. Affichez le résumé statistique
5. Listez les noms des colonnes et des lignes
6. Combien de voitures ont plus de 6 cylindres ?

Indice : `head()`, `tail()`, `dim()` (ou `nrow()` / `ncol()`), `str()`, `summary()`, `names()` (ou `colnames()`), `rownames()`.

▶ Solution

Exercice 5.4 : Explorer le jeu iris [Difficulté : Intermédiaire]

Énoncé : Avec le jeu de données `iris` :

1. Affichez les dimensions et la structure
2. Combien y a-t-il d'espèces différentes ? Lesquelles ?
3. Combien d'observations par espèce ?
4. Calculez la moyenne de `Sepal.Length` par espèce (utilisez `tapply()`)
5. Affichez les lignes où `Petal.Length > 6`
6. Quel est le pourcentage de valeurs uniques dans chaque colonne numérique ?

Indice : `unique()` ou `levels()` pour les espèces. `table()` pour le comptage. `tapply(valeurs, groupes, fonction)` applique une fonction par groupe.

▶ Solution

Exercice 5.5 : Gérer les valeurs manquantes [Difficulté : Intermédiaire]

Énoncé : Chargez le jeu de données `airquality` (intégré dans R) :

1. Combien de valeurs manquantes y a-t-il au total ?
2. Combien de NA par colonne ?
3. Quel pourcentage de lignes sont complètes (sans aucun NA) ?
4. Créez une version du data frame sans aucune ligne contenant des NA
5. Créez une version où les NA de `Ozone` sont remplacés par la médiane de `Ozone`
6. Comparez les moyennes d'Ozone avant et après imputation

Indice : `complete.cases()` retourne un vecteur logique. `colSums(is.na())` compte les NA par colonne. `median(x, na.rm = TRUE)` calcule la médiane en ignorant les NA.

▶ Solution

Exercice 5.6 : Exporter des données [Difficulté : Facile]

Énoncé :

1. Créez un résumé statistique d'`iris` par espèce (moyenne de chaque variable numérique)
2. Exportez ce résumé en CSV avec `write.csv()`
3. Exportez-le aussi en format tabulé avec `write.table()`
4. Sauvegardez l'objet R avec `saveRDS()` et rechargez-le avec `readRDS()`
5. Vérifiez que l'objet rechargé est identique

Indice : `aggregate()` avec la formule `. ~ Species` calcule une statistique pour chaque variable par groupe. `saveRDS()` sauvegarde un seul objet R.

▶ Solution

Module 6 : Manipulation de données

Exercice 6.1 : Sélection de colonnes [Difficulté : Facile]

Énoncé : Avec le jeu de données `mtcars` :

1. Sélectionnez uniquement les colonnes `mpg`, `cyl` et `hp`
2. Sélectionnez toutes les colonnes sauf `vs` et `am`
3. Sélectionnez les colonnes par position (colonnes 1, 3 et 5)
4. Sélectionnez les colonnes dont le nom contient la lettre "a" (utilisez `grep()`)

Indice : Sélection par nom : `df[, c("col1", "col2")]`. Exclusion : `df[, !names(df) %in% c("col1")]`. `grep()` cherche un motif dans les noms.

▶ Solution

Exercice 6.2 : Filtrage de lignes [Difficulté : Intermédiaire]

Énoncé : Avec `mtcars` :

1. Filtrez les voitures avec plus de 200 chevaux (`hp > 200`)
2. Filtrez les voitures à 4 cylindres ET transmission automatique (`am == 0`)
3. Filtrez les voitures dont la consommation est entre 20 et 30 mpg
4. Filtrez les voitures dont le nom contient "Merc" (utilisez `grep()` sur `rownames()`)
5. Utilisez `subset()` pour les voitures à 6 cylindres pesant moins de 3 tonnes

Indice : `grep("motif", rownames(df))` retourne les positions des lignes correspondantes. `subset()` est plus lisible que l'indexation classique.

▶ Solution

Exercice 6.3 : Tri de données [Difficulté : Facile]

Énoncé : Avec `mtcars` :

1. Triez par consommation (`mpg`) croissante
2. Triez par nombre de chevaux (`hp`) décroissant

3. Triez par nombre de cylindres croissant, puis par consommation décroissante
4. Affichez le top 5 des voitures les plus économes
5. Affichez le top 5 des voitures les plus puissantes

Indice : `order()` retourne les indices de tri. Pour un tri décroissant : `order(x, decreasing = TRUE)` ou `order(-x)` pour les numériques.

▶ Solution

Exercice 6.4 : Ajouter et transformer des colonnes [Difficulté : Intermédiaire]

Énoncé : Avec `mtcars` :

1. Ajoutez une colonne `kpl` (kilomètres par litre) : mpg x 0.4251
2. Ajoutez une colonne `poids_kg` : wt x 453.6 (wt est en milliers de livres)
3. Créez une colonne `categorie_conso` : "économe" si mpg > 25, "moyen" si 15-25, "gourmand" si < 15
4. Ajoutez une colonne `puissance_poids` : hp / wt (rapport puissance/poids)
5. Créez une colonne `transmission` : "automatique" si am == 0, "manuelle" si am == 1

Indice : Utilisez `ifelse()` imbriqués pour les catégories à 3 niveaux : `ifelse(cond1, val1, ifelse(cond2, val2, val3))`.

▶ Solution

Exercice 6.5 : Fusion de data frames (merge) [Difficulté : Intermédiaire]

Énoncé :

1. Créez un data frame `employes` : id (1:5), nom, departement
2. Créez un data frame `salaires` : id (1:6, incluant un id 6 inexistant dans employes), salaire, bonus
3. Effectuez un inner join (merge par défaut)
4. Effectuez un left join (garder tous les employés)
5. Effectuez un full join (garder toutes les lignes)

6. Fusionnez sur le nom du département avec un 3e data frame

Indice : `merge(x, y, by = "colonne")` fait un inner join. Ajoutez `all.x = TRUE` pour un left join, `all = TRUE` pour un full join.

▶ Solution

Exercice 6.6 : Agrégation de données [Difficulté : Intermédiaire]

Énoncé : Avec `mtcars` :

1. Calculez la consommation moyenne par nombre de cylindres avec `aggregate()`
2. Calculez la puissance moyenne ET l'écart-type par nombre de cylindres
3. Calculez le poids moyen par combinaison cylindres/transmission
4. Comptez le nombre de voitures par catégorie de cylindres
5. Trouvez la voiture la plus puissante dans chaque groupe de cylindres

Indice : `aggregate(formule, data, FUN)`. La formule est `variable ~ groupe`. Pour plusieurs variables : `cbind(v1, v2) ~ groupe`. Pour plusieurs groupes : `variable ~ g1 + g2`.

▶ Solution

Exercice 6.7 : Restructurer avec stack/unstack [Difficulté : Intermédiaire]

Énoncé :

1. Créez un data frame avec les notes de 3 matières pour 4 élèves
2. Utilisez `stack()` pour passer en format long (une seule colonne de notes)
3. Utilisez `unstack()` pour revenir au format large
4. Utilisez `reshape()` pour transformer le data frame de large en long, puis de long en large

Indice : `stack()` empile les colonnes numériques. `reshape()` est plus flexible avec les paramètres `direction`, `varying`, `v.names`, `timevar`, `idvar`.

▶ Solution

Exercice 6.8 : Manipulation avancée avec base R [Difficulté : Avancé]

Énoncé : Avec le jeu de données `airquality` :

1. Créez une colonne `mois_nom` avec les noms des mois en français (Mai, Juin, Juillet, Août, Septembre)
2. Créez une colonne `saison` : "printemps" (Mai), "été" (Juin-Août), "automne" (Septembre)
3. Calculez les statistiques descriptives (moyenne, médiane, sd) d'Ozone par mois (en ignorant les NA)
4. Identifiez le jour le plus chaud de chaque mois
5. Créez un sous-ensemble contenant uniquement les jours où Ozone > 100 et Temp > 90
6. Calculez la corrélation entre toutes les variables numériques (en supprimant les NA)

Indice : `month.name` contient les noms de mois en anglais. Créez un vecteur de correspondance pour le français. `cor()` avec `use = "complete.obs"` gère les NA.

▶ Solution

Module 7 : Visualisation graphique

Exercice 7.1 : Nuage de points (scatter plot) [Difficulté : Facile]

Énoncé : Avec `mtcars` :

1. Créez un nuage de points : poids (wt) en x, consommation (mpg) en y
2. Ajoutez un titre, des étiquettes d'axes en français
3. Colorez les points selon le nombre de cylindres
4. Ajoutez une droite de régression avec `abline()` et `lm()`
5. Ajoutez une légende

Indice : `plot(x, y)` crée le graphique. `col` pour la couleur, `pch` pour la forme. `abline(lm(y ~ x))` ajoute la droite de régression.

▶ Solution

Exercice 7.2 : Histogramme [Difficulté : Facile]

Énoncé : Avec `airquality` :

1. Créez un histogramme de la température (`Temp`)
2. Personnalisez : titre, couleur de remplissage, couleur de bordure
3. Ajustez le nombre de barres avec `breaks = 15`
4. Ajoutez une ligne verticale pour la moyenne et la médiane
5. Superposez une courbe de densité (normalisée)

Indice : `hist()` crée l'histogramme. Utilisez `freq = FALSE` pour des densités au lieu de fréquences, permettant de superposer `lines(density(x))`.

▶ Solution

Exercice 7.3 : Boîte à moustaches (boxplot) [Difficulté : Intermédiaire]

Énoncé : Avec `iris` :

1. Créez un boxplot de `Sepal.Length` pour chaque espèce
2. Personnalisez les couleurs (une par espèce)
3. Ajoutez un titre et des étiquettes en français
4. Orientez horizontalement
5. Identifiez les outliers (points aberrants) et affichez leurs valeurs

Indice : `boxplot(formule, data)` avec la formule `variable ~ groupe`. Le résultat d'un boxplot contient `$out` pour les outliers.

▶ Solution

Exercice 7.4 : Diagramme en barres (barplot) [Difficulté : Intermédiaire]

Énoncé :

1. Créez un tableau de fréquences du nombre de cylindres dans `mtcars`
2. Faites un barplot simple de ces fréquences
3. Créez un barplot empilé (stacked) : cylindres par type de transmission
4. Créez un barplot groupé (beside) pour la même combinaison

5. Ajoutez les valeurs numériques au-dessus des barres

Indice : `table()` pour les fréquences. `barplot()` avec une matrice crée des barres empilées. `beside = TRUE` pour des barres côte à côte. `text()` ajoute du texte.

▶ Solution

Exercice 7.5 : Multi-graphiques [Difficulté : Avancé]

Énoncé : Avec `airquality` :

1. Configurez la fenêtre pour afficher 4 graphiques (2x2) avec `par(mfrow)`
2. Graphique 1 : histogramme de `Ozone` (sans NA)
3. Graphique 2 : boxplot de `Temp` par mois
4. Graphique 3 : nuage de points `Wind` vs `Temp`
5. Graphique 4 : barplot des jours de données disponibles (non-NA) pour Ozone par mois
6. Remettez les paramètres graphiques par défaut

Indice : `par(mfrow = c(2, 2))` divise la fenêtre en 2 lignes et 2 colonnes. `par(mfrow = c(1, 1))` remet à la normale. `tapply()` peut compter les non-NA par mois.

▶ Solution

Exercice 7.6 : Personnalisation avancée [Difficulté : Avancé]

Énoncé : Créez un graphique professionnel avec `mtcars` :

1. Tracez mpg vs hp avec des points dont la taille varie selon le poids (wt)
2. Colorez selon le nombre de cylindres
3. Ajoutez un titre principal et un sous-titre
4. Personnalisez les marges avec `par()`
5. Ajoutez une grille de fond avec `grid()`
6. Annotez le point le plus puissant avec `text()`
7. Ajoutez un encadré informatif avec `rect()` et `text()`

Indice : Le paramètre `cex` contrôle la taille des points. `grid()` ajoute une grille. `text(x, y, label)` ajoute du texte. `rect(x1, y1, x2, y2)` dessine un rectangle.

▶ Solution